



CENTRO UNIVERSITARIO DE
CIENCIAS EXACTAS E INGENIERÍAS



Bases de Programación en Ensamblador

Programación de Bajo Nivel

Actividad 2

Alumno: Quijas Contreras Victor Manuel

Código: 220429771

Carrera: ICOM

Profesor: Meza Espinosa José Juan

Fecha: 01/02/2026

Índice

Índice	1
Tabla de ilustraciones	1
Código comentado.....	1
Preguntas	2
Conclusión	4
Bibliografía.....	4

Tabla de ilustraciones

Ilustración 1. Definición de variables.....	1
Ilustración 2. Código - 1ra sección.....	2
Ilustración 3. Código - 2da sección	2

Código comentado

```

01 org 100h          ; Indica al ensamblador que el código se cargara
02                  ; en la dirección 100h.
03
04 jmp inicio        ; Salta directo a donde se encuentra la etiqueta inicio.
05
06 msg0: db "Implementare un sumador de 2 numeros",0Dh,0Ah
07          db "cada numero es de 1 solo digito",0Dh,0Ah,24h
08          ; "db" Define Byte: reserva y almacena informacion en la memoria pero en
09          ; un solo byte.
10          ; Define el mensaje de bienvenida.
11          ; 0Dh retorno de carro y 0Ah salto de linea.
12          ; 24h es el caracter '$' y se necesita para indicar el fin
13          ; de cadena a la interrupcion 21h/AH=9h.
14
15 msg1: db "Dame el 1er numero: ",24h ; Solicita el primer numero.
16 msg2: db "Dame el 2do numero: ",24h ; Solicita el segundo numero.
17 msg3: db "El resultado es: ",24h    ; Mensaje para dar el resultado.
18 salto: db 0Dh,0Ah,24h              ; Cadena auxiliar para hacer un salto de linea en pantalla.
19 var1 db ?                          ; Reserva 1 byte de memoria no inicializado para guardar el primer operando.
20 var2 db ?                          ; el segundo operando y el resultado.
21 var3 db ?

```

Ilustración 1. Definición de variables

Bases de Programación en Ensamblador

```
23 inicio: ; A esta etiqueta salta el programa.
24 ; Imprimir mensaje de bienvenida.
25 mov ah,9h ; Carga la función 09h en AH que viene siendo
26 ; una interrupción para mostrar una cadena terminada en '$'.
27 mov dx,msg0 ; Pone la dirección de 'msg0' en el registro DX.
28 int 21h ; Llama a la interrupción del sistema DOS para imprimir la cadena.
29
30 ; Imprimir petición del primer número.
31 mov ah,9h ; Repite el proceso de imprimir las cadenas.
32 mov dx,msg1 ;
33 int 21h ;
34
35 ; Leer el primer carácter desde el teclado
36 mov ah,1h ; Carga la función 01h en AH que es la lectura de un carácter con eco en pantalla.
37 int 21h ; Espera la tecla del usuario y lo guarda en el registro AL en código ASCII.
38 sub al,30h ; Resta 30h (48 decimal) al código ASCII para convertirlo
39 ; a su valor numérico real.
40 mov var1,al ; Guarda el valor numérico obtenido en la variable 'var1'.
41
42 ; Imprimir salto de línea
43 mov ah,9h ; Carga la instrucción de salto de línea en el acumulador.
44 mov dx,salto ; Pone la dirección de 'salto' en DX.
45 int 21h ; Ejecuta la interrupción para bajar el cursor a una nueva línea.
46
47
```

Ilustración 2. Código - 1ra sección

```
48 ; -- Repite el proceso de imprimir y pedir datos --
49 mov ah,9h ;
50 mov dx,msg2 ;
51 int 21h ;
52
53 mov ah,1h ;
54 int 21h ;
55 sub al,30h ;
56 mov var2,al ;
57
58 mov al,var1 ;
59 mov bl,var2 ;
60 add al,bl ;
61 mov var3,al ;
62
63 mov ah,9h ;
64 mov dx,salto ;
65 int 21h ;
66
67 ; Imprimir mensaje de resultado
68 mov ah,9h ; Carga la instrucción de imprimir cadena en el acumulador.
69 mov dx,msg3 ; Pone la dirección de 'msg3' en DX.
70 int 21h ; Llama a la interrupción para imprimir "El resultado es: ".
71
72 ; Mostrar el dígito resultante en pantalla
73 mov ah,2h ; Carga la función 02h en el acumulador
74 ; interrupción para imprimir un solo carácter desde DL.
75 mov dl,var3 ; Copia el resultado numérico guardado en 'var3' al registro DL.
76 add dl,30h ; Le suma 30h (48 decimal) para reconvertir el número a su carácter ASCII correspondiente.
77 int 21h ; Llama a la interrupción para mostrar el carácter del resultado en pantalla.
78
79 ret ; Finaliza el programa y regresa el control al sistema operativo al DOS.
```

Ilustración 3. Código - 2da sección

Preguntas

1. ¿Por qué se utiliza `jmp inicio` al principio del programa?

Para saltar directo a la etiqueta señalada, en el caso del ejemplo, a la sección de código.

2. ¿Qué tipo de información contienen `msg0`, `msg1`, `msg2` y `msg3`?

Contienen cadenas de caracteres, cada carácter se guarda en un byte en formato ASCII.

3. ¿Qué significa `db`?

Significa "Define Byte" y se encarga de asignar y definir espacio en la memoria del tamaño de un byte.

4. ¿Por qué antes de `int 21h` se coloca el número de función en `AH`?

Porque se necesita cargar la instrucción en la parte alta del acumulador antes de ejecutar con la interrupción 21.

5. ¿Qué registro se utiliza para indicar la dirección del mensaje que se va a imprimir?

El registro de 16 bits DX.

6. ¿Por qué es necesario hacer sub al,30h?

Para transformar el carácter de código ASCII a su valor real.

7. ¿Qué diferencia existe entre el carácter '5' y el valor numérico 5?

El carácter se encuentra en código ASCII mientras que el valor numérico es su propio valor.

8. Si el usuario introduce '8', ¿qué valor hexadecimal tiene originalmente AL?

38h.

9. ¿En qué registro se realiza realmente la suma?

En la parte baja del acumulador

10. ¿Por qué el programa utiliza AL y BL en lugar de AX y BX?

Porque almacena caracteres de un solo dígito de 8 bits.

11. ¿Qué sucede si el usuario introduce a en lugar de un número?

El programa funcionará sin problema, pero operará con el valor del código ASCII de 'a' y eso generará un resultado que parezca incorrecto o algún otro tipo de carácter.

12. ¿Qué registro podríamos utilizar para detectar un acarreo después de ADD?

El registro de bandera acarreo.

13. ¿Por qué no podemos simplemente sumar directamente los caracteres que introduce el usuario?

Porque al estar en código ASCII el resultado no sería correcto, no estamos sumando valores sino ids.

14. ¿Dónde ocurre la conversión ASCII → número?

En la línea 39 y 55.

Conclusión

Comprender cuales son las instrucciones que se pueden aplicar en lenguaje ensamblador y cómo funciona el ciclo de carga y ejecución de estas mismas es fundamental para comenzar a programar a bajo nivel, la actividad nos muestra el uso de las instrucciones necesarias para entrada y salida de datos, que valores necesita la instrucción para ejecutarse e incluso el formato de conversión de valores tipo caracteres a su valor real. La finalidad de esta actividad es establecer las bases para realizar programas más complejos.

Bibliografía

- Espinosa, J. J. M. (s/f). *int 21h int 10h*. Recuperado <https://drive.google.com/drive/folders/13eTLM4CpLLdw8hrTZ96qEWDraWx0n3id>
- emu8086 documentation. (s/f). Github.io. Recuperado el 15 de septiembre de 2026, de <https://yassinebridi.github.io/asm-docs/>